

Universidad Tecnológica de Bolívar

UNIVERSIDAD TECNOLOGICA DE BOLIVAR

TALLER DEL SEGUNDO CORTE: DOCKER

Parte 2. Informe de práctica - Biblioteca API

Asignatura: Arquitectura de Software

Estudiante: Guillermo Andrés Rodríguez Munive

Profesor: Luis Giovanny Carreno Ortiz

16 de octubre de 2026

Taller del segundo corte: Docker

Parte 2. Informe de práctica - Biblioteca API
Asignatura: Arquitectura de Software
Estudiante: Guillermo Andrés Rodríguez Munive
Fecha de realización: 16 de octubre de 2026
Proyecto utilizado: https://github.com/carrenoutb/biblioteca-api

Introducción

En esta práctica utilicé Docker Desktop y Docker Compose para ejecutar una API de biblioteca desarrollada en Go, con el framework Gin y una base de datos MySQL. Seguí las indicaciones del repositorio, revisé los archivos de configuración y comprobé su funcionamiento mediante Swagger. Además, realicé tres operaciones: registrar un usuario, iniciar sesión y consultar los libros. En este informe presento los comandos utilizados, los resultados obtenidos y las capturas de mi computador.

Fase 1. Instalación y preparación del equipo

Cloné el repositorio del curso y entré a la carpeta de trabajo con estos comandos:

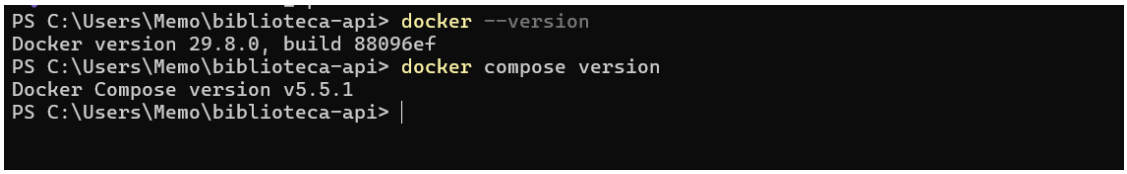
```
git clone https://github.com/carrenoutb/biblioteca-api.git
cd biblioteca-api
```

Primero tuve que solucionar un problema con Git, porque Windows no reconocía el comando git. Después, Docker Desktop no lograba iniciar su motor. Instalé el Subsistema de Windows para Linux (WSL), lo actualicé y pude ejecutar Docker Desktop. Finalmente, comprobé las versiones instaladas:

```
docker --version
docker compose version
```

La terminal mostró **Docker 29.8.0** y **Docker Compose v5.5.1**.

Evidencia 1. Versiones de Docker y Docker Compose.



Fase 2. Revisión de los archivos y configuración

Revisé los archivos principales del repositorio:

Elemento	Función en el proyecto
Dockerfile	Contiene las instrucciones para construir la imagen ejecutable de la API en Go.
docker-compose.yml	Configura la API, MySQL, los puertos, la red, el volumen y el orden de inicio.
.env.example	Proporciona la plantilla de variables que necesita la aplicación.
migrations/	Contiene las instrucciones SQL para preparar las tablas y datos de ejemplo.

En PowerShell creé el archivo local de variables y comprobé que se había generado:

```
Copy-Item .env.example .env
Test-Path .env
```

El segundo comando devolvió **True**, por lo que confirmé la presencia del archivo sin mostrar su contenido.

Evidencia 2. Verificación del archivo .env.



Variables de entorno

Variable	¿Para qué sirve?
DB_HOST	Identifica el servicio de la base de datos; aquí se utiliza mysql.
DB_PORT	Define el puerto interno de conexión a MySQL (3306).
DB_USER	Indica el usuario con el que se conecta la API.
DB_PASSWORD	Es la contraseña que usa ese usuario de MySQL.
DB_NAME	Establece el nombre de la base de datos (biblioteca).
JWT_SECRET	Define la clave utilizada para firmar y verificar tokens JWT.
JWT_EXPIRATION_HOURS	Establece la duración del token en horas.
APP_PORT	Define el puerto HTTP en el que escucha la API (8080).

El archivo .env no se adjunta a la entrega porque puede contener contraseñas y claves de autenticación.

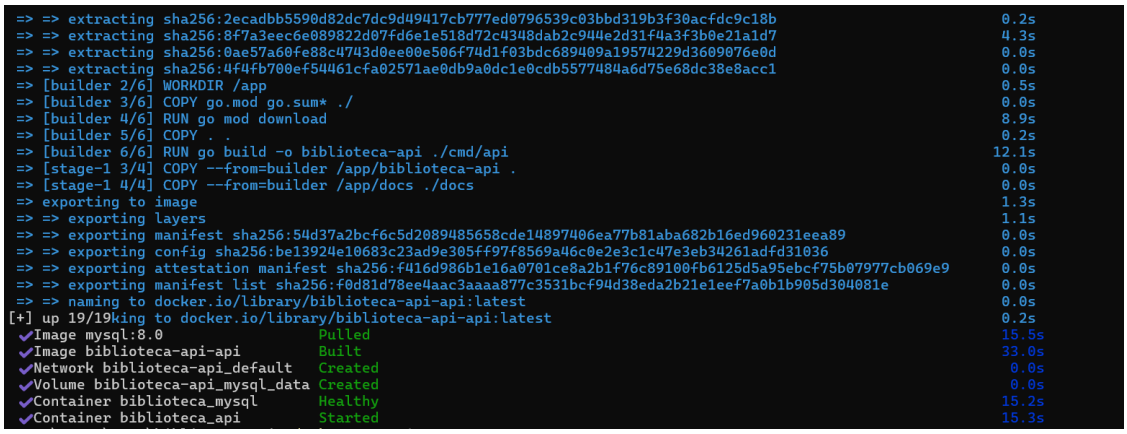
Fase 3. Construcción e inicio con Docker Compose

Desde la carpeta del proyecto ejecuté:

```
docker compose up --build -d
```

Este comando construyó la imagen de la API, descargó la imagen mysql:8.0, creó la red y el volumen del proyecto e inició los contenedores en segundo plano. La terminal mostró que la imagen de la API terminó de construirse, que **biblioteca_mysql estaba Healthy** y que **biblioteca_api estaba Running**.

Evidencia 3. Descarga de imágenes y construcción.



Evidencia 4. Servicios iniciados.



El comando generó una advertencia porque la propiedad version de docker-compose.yml está obsoleta para la versión instalada. Docker la ignoró y el proyecto continuó ejecutándose.

Fase 4. Verificaciones de funcionamiento

4.1. Estado de los contenedores

Para comprobar los servicios ejecuté:

```
docker compose ps
```

El resultado mostró biblioteca_api activo en el puerto **8080**, y biblioteca_mysql activo y saludable en el puerto **3306**.

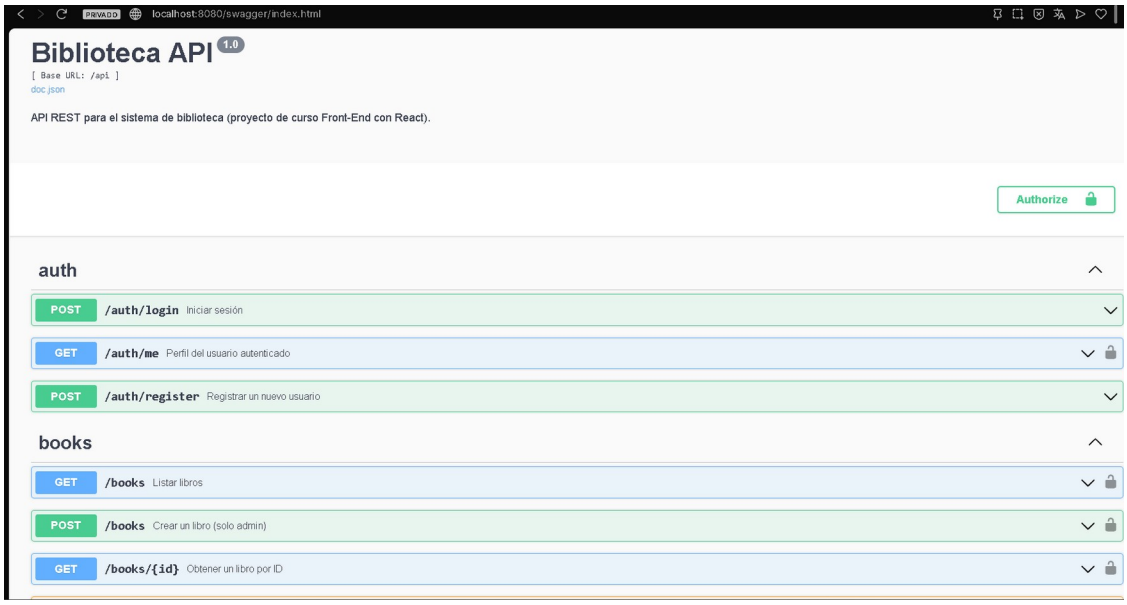
Evidencia 5. Estado y puertos de los contenedores.



4.2. Swagger y estado de salud

Abrí la documentación interactiva en http://localhost:8080/swagger/index.html. Allí aparecieron los grupos de operaciones auth y books, lo que permitió realizar las pruebas desde el navegador.

Evidencia 6. Swagger funcionando.



Después consulté http://localhost:8080/health y recibí {"status":"ok"}. Esta respuesta confirmó que el servidor HTTP estaba atendiendo solicitudes.

Evidencia 7. Respuesta del endpoint /health.



4.3. Registro de usuario

En Swagger ejecuté POST /api/auth/register con un usuario de prueba. El servidor respondió con **HTTP 201 Created**, generó un usuario con rol **student** y entregó un token JWT. La captura que adjunto oculta el token completo por seguridad.

Evidencia 8. Usuario registrado correctamente (HTTP 201).



4.4. Inicio de sesión

Ejecuté POST /api/auth/login con las credenciales del usuario de prueba. El resultado fue **HTTP 200 OK** y un token JWT válido para acceder a rutas protegidas.

Evidencia 9. Inicio de sesión correcto (HTTP 200).



4.5. Consulta del catálogo de libros

Al intentar ejecutar por primera vez GET /api/books, el servidor respondió **401 Unauthorized** con el mensaje token no proporcionado, porque el valor enviado en el encabezado Authorization no tenía el prefijo requerido. Corregí la autorización escribiendo Bearer seguido de un espacio y del token. La nueva petición respondió **HTTP 200 OK** y devolvió datos de libros.

Entre los títulos de la respuesta aparecieron *Cien años de soledad*, *Clean Code*, *El principito* y *The Pragmatic Programmer*. Se recibieron campos como title, author, isbn, category, total_copies y available_copies.

Evidencia 10. Consulta de libros autenticada (HTTP 200).



4.6. Revisión de los registros de los servicios

Para revisar la actividad ejecuté:

```
docker compose logs --tail=40 api mysql
```

Los registros muestran respuestas **201** para el registro, **200** para el inicio de sesión, **401** para el primer intento de consulta y **200** para la consulta de libros después de corregir la autorización. También muestran que MySQL quedó listo para conexiones.

Observación técnica: durante la inicialización de los datos de ejemplo, MySQL registró un **ERROR 1064 (42000)** en migrations/001_init.sql, correspondiente a una instrucción SQL con sintaxis incorrecta. No se modificó la migración original del profesor. El error no impidió que MySQL quedara operativo ni que las operaciones verificadas fueran exitosas, pero no se puede afirmar que todas las instrucciones de carga inicial terminaron correctamente.

Evidencia 11. Registros reales de la API y MySQL.



Problemas encontrados y soluciones

1. Docker Desktop no podía iniciar. El comando de Compose inicialmente devolvía Docker Desktop is unable to start. Para resolverlo, instalé/actualicé WSL y reinicié el entorno. Después Docker Desktop inició y permitió levantar los contenedores.

2. Error de autorización en la consulta. El primer GET /api/books devolvió 401 Unauthorized porque el encabezado de autorización no tenía el formato que la API esperaba. Agregué el prefijo Bearer al JWT y la siguiente petición respondió 200 OK.

3. Error SQL durante la carga de ejemplos (pendiente). Los logs muestran un error de sintaxis en migrations/001_init.sql. No se intervino el archivo fuente y se dejó registrada la observación; la API siguió funcionando en las pruebas realizadas.

Preguntas de comprensión

1. ¿Qué diferencia hay entre una imagen y un contenedor? Utiliza un ejemplo del proyecto.

La imagen es una plantilla que incluye lo necesario para ejecutar un programa, mientras que el contenedor es una instancia de esa imagen puesta en funcionamiento. Por ejemplo, mysql:8.0 es la imagen de MySQL que utiliza el proyecto y biblioteca_mysql es el contenedor que se crea al iniciar el servicio. De forma similar, la imagen de la API se construye desde el Dockerfile, y de ella se crea biblioteca_api.

2. ¿Qué hace Docker Compose que no harías con docker run por separado?

Compose permite definir y administrar todos los servicios del proyecto desde un solo archivo. En lugar de escribir manualmente varios comandos docker run, se configuran en el YAML los servicios, puertos, volúmenes y dependencias. En esta práctica, Compose crea los contenedores de MySQL y la API, los conecta en una red interna y espera que MySQL esté saludable antes de iniciar la API. Con docker run también se puede lograr, pero habría que configurar y coordinar esos elementos por separado.

3. ¿Por qué la API encuentra la base de datos con DB_HOST=mysql y no con 127.0.0.1?

Porque mysql es el **nombre del servicio** dentro de la red creada por Docker Compose. Desde el contenedor de la API, ese nombre se resuelve a la dirección del contenedor de la base de datos. En cambio, 127.0.0.1 hace referencia al propio contenedor de la API, no al contenedor de MySQL. Por eso la conexión entre ambos servicios utiliza DB_HOST=mysql.

4. ¿Qué borra exactamente docker compose down -v y cuándo lo usarías?

Detiene y elimina los contenedores y la red creados por Compose y, además, elimina los **volúmenes con nombre declarados en el archivo Compose** y los volúmenes anónimos asociados. Para este proyecto elimina el volumen mysql_data, por lo que se pierden los datos persistidos de la base de datos, incluidos usuarios o préstamos añadidos durante la práctica. No elimina por defecto las imágenes ni el código fuente. Lo utilizaría cuando necesito reiniciar el proyecto con una base de datos vacía y volver a cargar los datos iniciales. Es importante no ejecutarlo si necesito conservar la información.

5. ¿Qué pasaría si cambias DB_PASSWORD en .env pero no en docker-compose.yml? ¿Y al revés?

El archivo .env configura la contraseña con la que la **API intenta conectarse**, mientras que el docker-compose.yml usa MYSQL_PASSWORD para definir la contraseña inicial del usuario de MySQL. Si cambio solo DB_PASSWORD, la API enviará una clave distinta y la conexión normalmente fallará. Si cambio solo MYSQL_PASSWORD en Compose y la base de datos se crea desde cero, MySQL configurará esa nueva contraseña y la API seguirá usando la anterior, por lo que también fallará la autenticación.

Hay un detalle adicional: cuando MySQL **ya tiene datos en el volumen mysql_data**, cambiar MYSQL_PASSWORD en Compose no actualiza automáticamente la contraseña del usuario existente. Para resolver una diferencia de credenciales deben coordinarse los valores de ambos servicios y, si la base ya fue creada, actualizar realmente la contraseña dentro de MySQL o recrear el volumen **solo si se pueden perder los datos**.

Conclusión

La práctica me permitió comprobar cómo Docker facilita ejecutar dos servicios relacionados, la API y la base de datos, sin tener que instalar Go o MySQL directamente en Windows. La construcción y el inicio de contenedores fueron correctos, y las pruebas en Swagger confirmaron que podía registrar un usuario, iniciar sesión y consultar información utilizando un token JWT. También aprendí a revisar los registros y a diferenciar entre advertencias que no detienen el sistema y errores que deben documentarse para revisarlos posteriormente.

Referencias

1. carrenoutb. (2026). *Biblioteca API*. GitHub. https://github.com/carrenoutb/biblioteca-api
2. carrenoutb. (2026). *Taller del 2do corte - Docker*. GitHub. https://github.com/carrenoutb/web-frontend-course/blob/main/taller-2do-corte-docker.md
3. Docker. (s. f.). *Docker Compose*. https://docs.docker.com/compose/
4. Docker. (s. f.). *Persisting container data*. https://docs.docker.com/engine/storage/volumes/